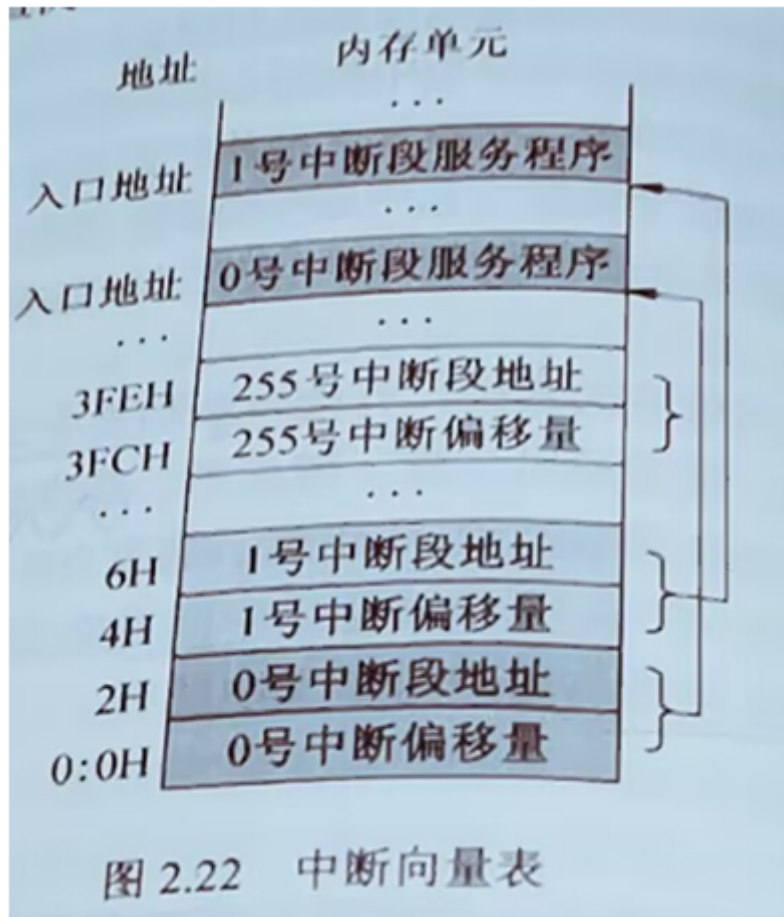


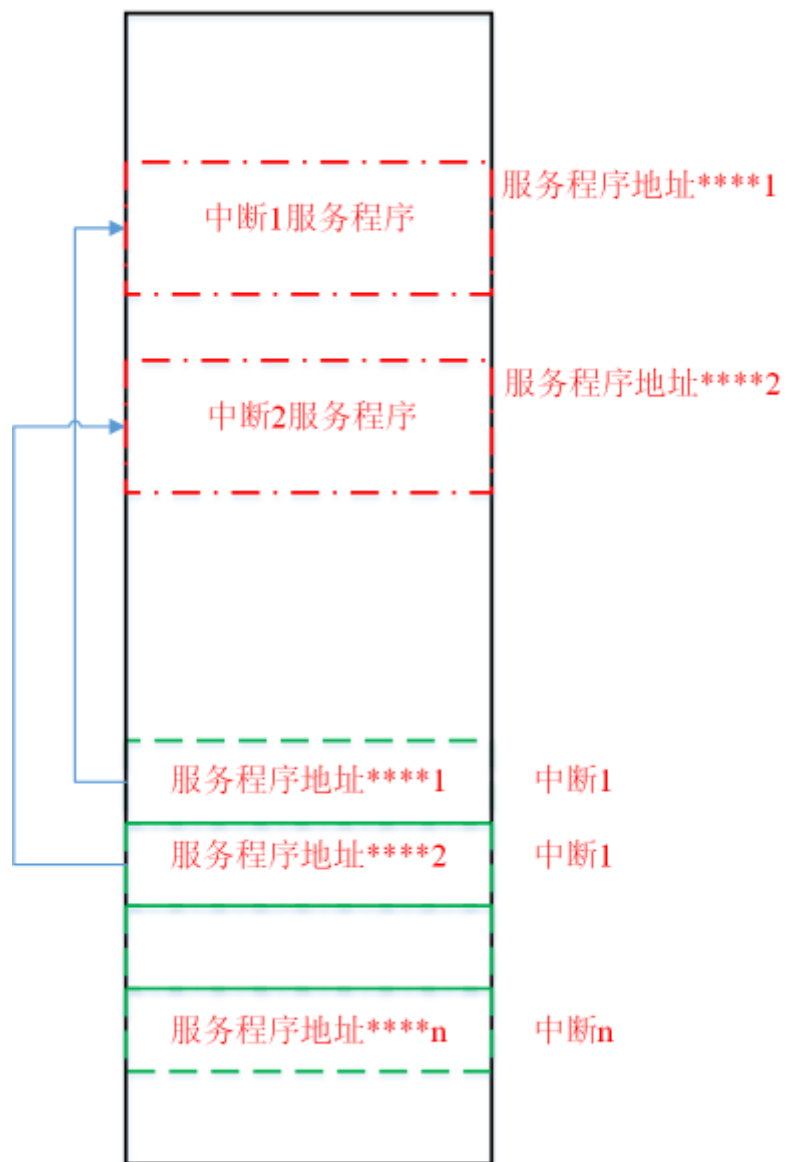
简答题

1.简述实模式下中断的原理，画图展示中断向量的结构。

中断就是CPU暂停当前程序的执行，转而执行处理紧急事务的程序，并在该事务处理完成后能自动恢复执行原先程序的过程。

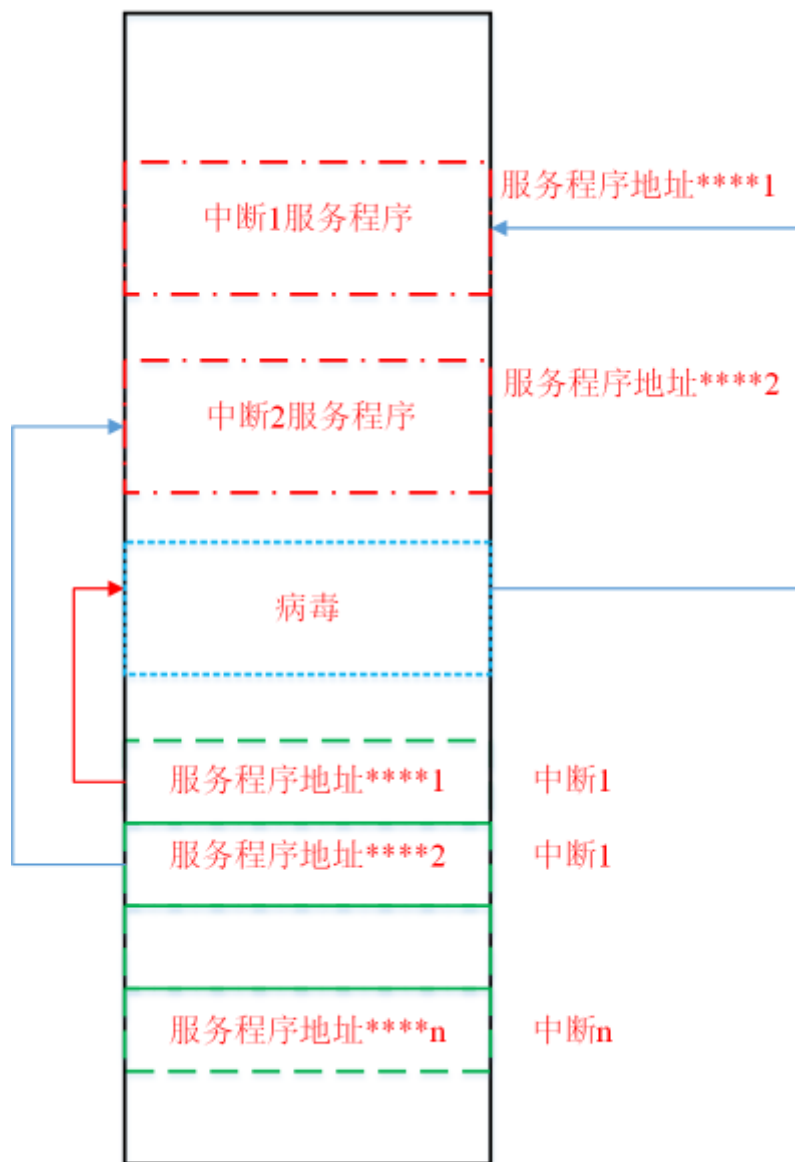
称引起紧急事务的事件为中断源，称处理紧急事务的程序为中断服务程序或中断处理程序。





2.简述引导型病毒中中断劫持、病毒代码激活的原理。

中断劫持：病毒通过修改中断向量表中的中断处理程序地址，使得当中断发生时，将中断服务程序劫持为病毒程序，控制权被转移到病毒的代码中，从而实现对系统的控制和感染。



病毒代码激活原理：利用一些常规的中断需求，在代码产生中断时，就可以劫持中断服务程序，跳转到病毒代码部分，激活病毒。

3.简述病毒重定位的原因以及实现的原理。

原因：

- 1.正常程序在源程序编译时，变量在内存中的位置都被计算好了，在装入内存时，系统不会为它重定位。
- 2.而病毒在感染HOST程序时，是将已编译好的代码附在HOST上，此时病毒代码中的变量地址并没有随着在内存位置改变而发生改变，故需要对病毒进行重定位操作。

原理：

当执行call指令时，它会先将返回地址压入堆栈，再将IP置为call语句所指向的地址。在被调子程序遇到ret指令后，就会将堆栈顶端的地址弹出来，并将其置入IP中，实现返回。

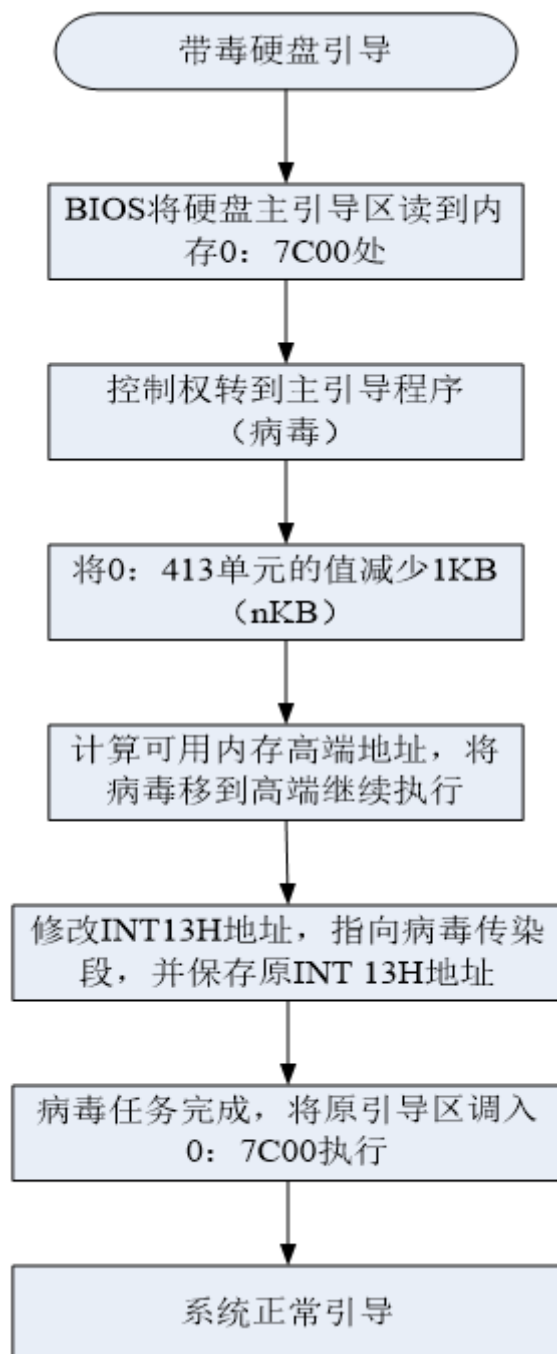
```
VStart:
...
call delta ; 这条语句执行之后,堆栈顶端为delta在内存中的真正地址
delta:
    pop    ebp ; 这条语句将delta在内存中的真正地址存放在ebp寄存器中
    sub    ebp, offset delta-vstart ; 得到一个偏移差,变量加上该值进行重定位
    lea    eax, [offset var2+ebp] ; 此时eax中存放着var2在内存中的真实地址
```

4. 引导型病毒的概念和基本原理

概念: 指专门感染主引导扇区和引导扇区的计算机病毒。感染主引导区的病毒称作MBR病毒; 感染引导区的病毒称作DBR病毒。

基本原理:

1. 通过感染主引导扇区和引导扇区, 在启动系统时即获取控制权。
2. 引导型病毒在感染主引导扇区/引导扇区时, 将被感染扇区的“内容”写入其他扇区, 并在FAT中标示该扇区所在簇为坏簇。
3. 为能在机器运行过程中能实施感染, 通过修改内存大小, 截取高端内存, 实现常驻内存。



5.引导型病毒的触发

获取控制权、驻留内存：引导型病毒在获取控制权后，修改INT 13H入口地址使其指向病毒中断服务程序，处于可激活态

激活：当系统/用户进行磁盘读写时调用INT 13H，激活病毒。

感染：病毒被激活后，可根据感染条件实施感染、根据爆发破坏条件破坏系统并表现自己。

6.引导型病毒的几个技术要点与特点

驻留内存：一般采用修改0: 413地址内的值的方法，但不够隐蔽。

隐形技术：当病毒驻留时，读写引导区均对原引导区操作，好像没有病毒一样

机密技术：一般加密分区表，使无毒盘启动时无法读取硬盘

优点：隐蔽性强、兼容性强

缺点：传染速度慢，一定要用带毒软盘启动才能传到硬盘；杀毒容易，只需改写引导区即可。

7.简述PE文件型病毒获取Kernel32.dll基址的原因和方法。

原因：PE病毒需要调用API函数实现某些功能，然而普通PE程序里面有一个导入函数节，通过导入节获取API函数地址，并不需要获取Kernel32.dll基址，而PE病毒只有代码节，对API函数的调用需要首先找到其在相应DLL中的地址。

方法：

1.利用程序的返回地址，在其附近搜索Kernel 32模块基址。

系统打开一个可执行文件时，会先调用Kernel32.dll中的createprocess函数；该函数在完成装载应用程序后，会先将一个返回地址压入到堆栈顶端，然后转向执行刚才装载的应用程序。我们可以获取该返回地址，再利用页对齐机制获取Kernel32的基址。

2.对相应操作系统分别给出固定的Kernel32模块的基址。

对于同一版本的Windows操作系统，Kernel32模块的地址是固定，甚至一些API函数的大概位置都是固定的。

8.PE文件型病毒如何获取Windows API函数地址。

1.在得到Kernel32.DLL的模块地址后，就可以在该模块中的导出表中搜索所需要的API地址

2.利用Kernel32中的两个函数GetProcAddress以及LoadLibrary，就可以获取所有其他api函数地址。

3.

获取API函数地址

■ 已知API函数序号搜索API函数地址

- 定位PE文件头
- 从PE文件头中的可选文件头中取出数据目录表的第一个数据目录，得出导出函数的地址。
- 从导出表的Base字段取得起始序号
- 将需要查找的到处序号减去起始序号，得到函数入口地址表中的索引号。
- 验证索引号的合法性
- 用该索引值在AddressOfFunctions字段指向的地址表取出相应的项目。

获取API函数地址

■ 已知函数名查找入口地址

- 定位PE文件头
- 从PE文件头中的可选文件头中取出数据目录表的第一个数据目录，得出导出函数的地址。
- 从AddressOfNames字段指向的函数名地址表的第一项开始逐一查找与其对应项。
- 通过在AddressOfNames中匹配项的索引值在AddressOfNameOrdinals指向的数组中查找索引值。
- 以该索引值在AddressOfFunctions中获取对应的RVA。

9.内存映射文件的原理和如何读写文件

原理：

内存映射文件提供了一组独立的函数，使应用程序能够通过内存指针像访问内存一样对磁盘上的文件进行访问。

内存映射文件函数将磁盘上的文件的全部或者部分映射到进程虚拟地址空间的某个位置，以后对文件内容的访问就如同在该地址区域内直接对内存访问一样。

如何读写文件：

- a.调用CreateFile函数打开想要映射的HOST程序，返回文件句柄hFile。
- b.调用CreateFileMapping函数生成一个建立基于HOST文件句柄hFile的内存映射对象，返回内存映射对象句柄hMap。
- c.调用MapViewOfFile函数将整个文件映射到内存中，得到指向内存的第一个字节的指针pMem。
- d.用指针pMem对整个HOST文件进行操作，实施病毒感染。
- e.调用UnMapViewFile函数解除文件映射，传入参数是pMem
- f.调用CloseHandle来关闭内存映射文件，传入参数是hMap
- g.调用CloseHandle来关闭HOST文件，传入参数是hFile。

10.病毒感染PE文件的基本步骤

- a.判断目标文件开始的两个字节是否为“MZ”
- b.判断PE文件标记“PE”
- c.判断感染标记，如果已被感染过则跳出继续执行HOST程序，否则实施感染
- d.获得Directory的个数，每个目录信息占8个字节
- e.得到节表起始位置
- f.得到目前最后节表的末尾偏移
- g.写入新节表:

写入节名;写入节的实际字节数;写入新节在内存中的开始偏移地址，同时可以计算出病毒入口位置;本节在内存中的开始偏移地址=上节在内存中的开始偏移地址+ (上节大小/节对齐+1) *节对齐;写入本节在文件中的对齐后的大小;写入本节在文件中的开始位置;修改映像文件头中的节表数目;修改AddressOfEntryPoint，同时保存旧的AddressOfEntryPoint，以便返回HOST继续执行;更新SizeOfImage (内存中整个映像尺寸=原SizeOfImage +病毒节经过内存节对齐后的大小) ;写入感染标记;写病毒代码到新添加的节中;将当前文件位置设为文件尾

11.简述PE文件基本的节表免疫和节间免疫方法。

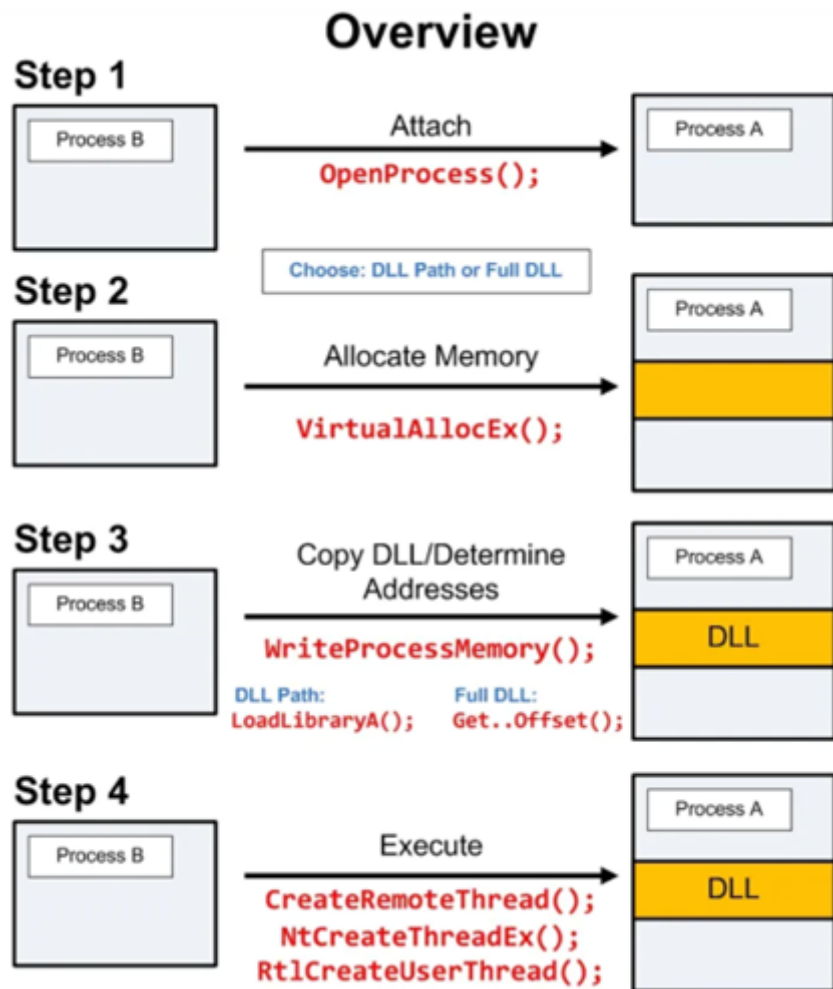
节表免疫：将节表项每一项往内存对齐的最后位置移动，再扩大OptionalHeader的size。

节间免疫：将每一个节表项中virtualsize的值改为sizeofrawdata的值，防止病毒写入节与节之间的碎片

12.简述通过CreateRemoteThread函数进行Dll注入的过程。

加载步骤：

- (1) .用VirtualAllocEx函数在目标进程的地址空间中分配一块足够大的内存用于保存被注入的dll的路径。
- (2) .用WriteProcessMemory函数把本进程中保存dll路径的内存中的数据拷贝到第（1）步得到的目标进程的内存中。
- (3) .用GetProcAddress函数获得LoadLibraryW函数的起始地址。LoadLibraryW函数位于Kernel32.dll中。
- (4) .用CreateRemoteThread函数让目标进程执行LoadLibraryW来加载被注入的dll。函数结束将返回载入dll后的模块句柄。



卸载步骤:

- (5).用VirtualFreeEx释放第(1)步开辟的内存。
- (6).用GetProcAddress函数获得FreeLibrary函数的起始地址。FreeLibrary函数位于Kernel32.dll中。
- (7).用CreateRemoteThread函数让目标进程执行FreeLibrary来卸载被注入的dll。(其参数是第(4)步返回的模块句柄)。

13.简述用CreateProcess对子进程注入dll的过程，并解释ShellCode的含义。

- (1) .创建一个挂起的子进程作为目标进程；
- (2) .准备一份预先设计好的ShellCode(应具有上面所述的基 本功能)；
- (3) .用VirtualAllocEx在目标进程中调拨一块可执行的内存；
- (4) .以分配的内存为基准修改ShellCode的汇编代码引用的地址和数据；
- (5) .用WriteProcessMemory函数将修改完毕的ShellCode复制目标进程在第3步分配的内存中；
- (6) .修改目标进程的主线程的EIP指向第3步分配的内存的首地址；
- (7) .恢复目标进程的主线程。

Shellcode的含义:

Shellcode是一段精心编写的汇编代码（机器码），通常用于实现特定功能。

14.简述调试hook的主要原理和过程。

流程：

- (1) .对想要钩取的进程进行附加操作(DebugActiveProcess)，使之成为被调试者。
- (2) .将要钩取的API的起始地址的第一个字节修改为0xcc(或者使用硬件断点)。
- (3) .当调用目标API的时候，控制权就转移到调试器进程。
- (4) .执行需要的操作。
- (5) .脱钩，将API 函数的第一个字节恢复。
- (6) .运行相应的API。
- (7) .再次修改为0xCC，为了继续钩取。
- (8) .控制权返还被调试者。

15.简述蠕虫与病毒的主要区别。

属性	病毒	蠕虫
存在形式	寄生	独立个体
复制机制	插入到宿主程序中	自身的拷贝
传染机制	宿主程序运行	系统存在漏洞
搜索机制	主要针对本地文件	主要针对网络上的其他计算机
触发传染	计算机使用者	程序自身
影响重点	文件系统	网络性能、系统性能
计算机使用者角色	病毒传播中的关键环节	无关
防治措施	从宿主程序中摘除	为系统打补丁
对抗主体	计算机使用者、反病毒厂商	系统提供商、网络管理员